Editorial workflow improvement

What I'd ask or observe first

Before building anything, I'd watch one editor do the actual workflow on a real article. Not a summary of it, not a Slack thread describing it. I'd sit next to them or screen share and take notes on every step.

The questions I'd want answers to:

- Which tools do they paste into? What order?
- What specifically gets copied each time? Title only? Title plus URL? Image plus caption?
- Where do mistakes actually happen, the copy step, the formatting step, or something else?
- How often does this happen in a day? Per editor?
- What happens when a mistake makes it through? How is it caught?

The last question matters most. If mistakes are caught before publishing, the cost is only time. If they reach readers, the cost is credibility. That changes the priority.

The smallest useful improvement

The simplest possible fix, and the one I'd default to if I only had one day: a small internal tool, a single page with three form fields and a copy button. The editor pastes the article URL. The tool fetches the title, the canonical URL, and the featured image, then puts each into its own copyable block. One click per block. That's it.

No API integrations. No backend sync. Just a page that takes a URL and hands back clean, pre-formatted pieces.

The simpler alternative

If even that is too much for one day, the plainest possible version is a browser bookmarklet. Drag it to your bookmarks bar. When you're on the article page, click it. A small panel appears with the title, URL, and image already in copyable blocks. Editors can install it in five seconds, no deployment needed.

The bookmarklet is the version I'd ship first because it doesn't touch any server, doesn't need anyone else's approval, and can be built and tested in a few hours.

Why I'd pick the bookmarklet

Because it's the fastest path to a real thing editors can use today. A hosted tool needs deployment, access control, and support. A bookmarklet lives on the editor's browser and only runs when they click it. That's the right amount of scope for one engineering day.

What remains under editorial control

Everything that actually matters. The editor picks which article, which image, which fields to copy. The tool never publishes anything, never changes content, never sends data anywhere. It just makes the copy step faster and less error prone. All editorial judgment stays with the editor.

How I'd test it with one editor

I'd pick one editor, sit with them for an hour, and have them use it on five real articles. I'd watch where they hesitate, where they click wrong, and what they say out loud. The goal isn't to see if it works, it's to see where it's annoying.

How I'd judge success

Two numbers. Time per article before and after. Errors before and after. If the editor saves a few minutes per article and makes fewer mistakes, it works. If either number doesn't move, it doesn't. I'd also ask the editor directly: would you keep using this tomorrow if I didn't ask you to?

A similar improvement from my own work

I built a small internal form generator for a client who was manually rewriting the same customer intake questions across multiple pages. Instead of filling the same blocks in each time, they could enter their answers once and the tool produced the finished text for each destination. The time per page dropped from about fifteen minutes to under two, and the copy-paste errors disappeared. What I learned is that the smallest useful version of a tool is usually one screen with one job. The temptation is to make it a platform. The discipline is to make it a button.